

```
package main

import (
    "os"

    "eqrcp/cmd"
)

func main() {
    if err := cmd.Execute(); err != nil {
        os.Exit(1)
    }
}

package application

type Flags struct {
    Quiet          bool
    KeepAlive      bool
    ListAllInterfaces bool
    Port           int
    Path           string
    Interface      string
    Bind           string
    FQDN           string
    Zip            bool
    Config         string
    Browser        bool
    Secure         bool
    TlsCert        string
    TlsKey         string
    Output         string
    Reversed       bool
}

type App struct {
    Flags Flags
    Name  string
}

func New() App {
    return App{
        Name:  "eqrcp",
        Flags: Flags{},
    }
}

package cmd

import (
    "eqrcp/application"
    "github.com/spf13/cobra"
)
```

```

)

var app application.App

func init() {
    app = application.New()
    rootCmd.AddCommand(sendCmd)
    rootCmd.AddCommand(receiveCmd)
    rootCmd.AddCommand(chatCmd)
    rootCmd.AddCommand(configCmd)
    rootCmd.AddCommand(versionCmd)
    rootCmd.AddCommand(completionCmd)
    rootCmd.AddCommand(desktopCmd)
    desktopCmd.AddCommand(desktopShareCmd)
    desktopCmd.AddCommand(desktopReceiveCmd)
    desktopCmd.AddCommand(desktopChatCmd)
    desktopCmd.AddCommand(desktopInstallCmd)
    desktopCmd.AddCommand(desktopStatusCmd)
    desktopCmd.AddCommand(desktopStartupEnableCmd)
    desktopCmd.AddCommand(desktopStartupDisableCmd)
    desktopCmd.AddCommand(desktopStartupStatusCmd)
    desktopCmd.AddCommand(desktopUninstallCmd)
    desktopCmd.AddCommand(desktopAgentCmd)
    desktopCmd.AddCommand(desktopAgentStartCmd)
    desktopCmd.AddCommand(desktopAgentRestartHelperCmd)
    desktopCmd.AddCommand(desktopAgentHistoryClearCmd)
    desktopCmd.AddCommand(desktopAgentStopCmd)
    desktopCmd.AddCommand(desktopAgentStopCurrentCmd)
    desktopCmd.AddCommand(desktopAgentStatusCmd)
    desktopCmd.AddCommand(desktopAgentOpenCmd)
    desktopCmd.AddCommand(desktopAgentOpenCurrentCmd)
    configCmd.AddCommand(migrateCmd)
    desktopAgentCmd.Flags().BoolP("background", "B", false, "start the desktop agent in the backgrou
    desktopAgentStartCmd.Flags().BoolP("background", "B", false, "start the desktop agent in the bac
    // Global command flags
    rootCmd.PersistentFlags().BoolVarP(&app.Flags.Quiet, "quiet", "q", false, "only print errors")
    rootCmd.PersistentFlags().BoolVarP(&app.Flags.KeepAlive, "keep-alive", "k", false, "keep server
    rootCmd.PersistentFlags().BoolVarP(&app.Flags.ListAllInterfaces, "list-all-interfaces", "l", fal

```

```

// Receive command flags
receiveCmd.PersistentFlags().StringVarP(&app.Flags.Output, "output", "o", "", "output directory
}

// The root command ('eqrcp') is like a shortcut of the 'send' command
var rootCmd = &cobra.Command{
    Use:           "eqrcp",
    Args:          cobra.MinimumNArgs(1),
    RunE:          sendCmdFunc,
    SilenceErrors: true,
    SilenceUsage:  true,
}

// Execute the root command
func Execute() error {
    if err := rootCmd.Execute(); err != nil {
        rootCmd.PrintErrf("Error: %v\nRun 'eqrcp help' for help.\n", err)
        return err
    }
    return nil
}
package cmd

import (
    "fmt"

    "eqrcp/body"
    "eqrcp/config"
    "eqrcp/logger"
    "eqrcp/qr"
    "github.com/eiannone/keyboard"

    "eqrcp/server"
    "github.com/spf13/cobra"
)

func sendCmdFunc(command *cobra.Command, args []string) error {
    log := logger.New(app.Flags.Quiet)
    body, err := body.FromArgs(args, app.Flags.Zip)
    if err != nil {
        return err
    }
    cfg, err := config.New(app)
    if err != nil {
        return err
    }
    srv, err := server.New(&cfg)
    if err != nil {
        return err
    }
}

```

```

// Sets the body
srv.Send(body)
log.Print('Scan the following URL with a QR reader to start the file transfer, press CTRL+C or "q"
log.Print(srv.SendURL)
if err := qr.RenderString(srv.SendURL, cfg.Reversed); err != nil {
    return err
}
if app.Flags.Browser {
    if err := srv.DisplayQR(srv.SendURL); err != nil {
        return err
    }
}
if err := keyboard.Open(); err == nil {
    defer func() {
        keyboard.Close()
    }()
    go func() {
        for {
            char, key, _ := keyboard.GetKey()
            if string(char) == "q" || key == keyboard.KeyCtrlC {
                srv.Shutdown()
            }
        }
    }()
} else {
    log.Print(fmt.Sprintf("Warning: keyboard not detected: %v", err))
}
if err := srv.Wait(); err != nil {
    return err
}
return nil
}

var sendCmd = &cobra.Command{
    Use:     "send",
    Short:   "Send a file(s) or directories from this host",
    Long:    "Send a file(s) or directories from this host",
    Aliases: []string{"s"},
    Example: `# Send /path/file.gif. Webserver listens on a random port
eqrcp send /path/file.gif
# Shorter version:
eqrcp /path/file.gif
# Zip file1.gif and file2.gif, then send the zip package
eqrcp /path/file1.gif /path/file2.gif
# Zip the content of directory, then send the zip package
eqrcp /path/directory
# Send file.gif by creating a webserver on port 8080
eqrcp --port 8080 /path/file.gif
`,
    Args: cobra.MinimumNArgs(1),

```

```

RunE: sendCmdFunc,
}
package cmd

import (
    "fmt"

    "eqrcp/config"
    "eqrcp/logger"
    "eqrcp/qr"
    "eqrcp/server"
    "github.com/eiannone/keyboard"
    "github.com/spf13/cobra"
)

func receiveCmdFunc(command *cobra.Command, args []string) error {
    log := logger.New(app.Flags.Quiet)
    // Load configuration
    cfg, err := config.New(app)
    if err != nil {
        return err
    }
    // Create the server
    srv, err := server.New(&cfg)
    if err != nil {
        return err
    }
    // Sets the output directory
    if err := srv.ReceiveTo(cfg.Output); err != nil {
        return err
    }
    // Prints the URL to scan to screen
    log.Print('Scan the following URL with a QR reader to start the file transfer, press CTRL+C or "q"')
    log.Print(srv.ReceiveURL)
    // Renders the QR
    if err := qr.RenderString(srv.ReceiveURL, cfg.Reversed); err != nil {
        return err
    }
    if app.Flags.Browser {
        if err := srv.DisplayQR(srv.ReceiveURL); err != nil {
            return err
        }
    }
    if err := keyboard.Open(); err == nil {
        defer func() {
            keyboard.Close()
        }()
        go func() {
            for {
                char, key, _ := keyboard.GetKey()

```

```

        if string(char) == "q" || key == keyboard.KeyCtrlC {
            srv.Shutdown()
        }
    }()
} else {
    log.Print(fmt.Sprintf("Warning: keyboard not detected: %v", err))
}
if err := srv.Wait(); err != nil {
    return err
}
return nil
}

var receiveCmd = &cobra.Command{
    Use:     "receive",
    Aliases: []string{"r"},
    Short:   "Receive one or more files",
    Long:    "Receive one or more files. The destination directory can be set with the config wizard, o
Example: '# Receive files in the current directory
eqrcp receive
# Receive files in a specific directory
eqrcp receive --output /tmp
',
    RunE: receiveCmdFunc,
}

package cmd

import (
    "fmt"

    "eqrcp/config"
    "eqrcp/logger"
    "eqrcp/qr"
    "eqrcp/server"
    "github.com/eiannone/keyboard"
    "github.com/spf13/cobra"
)

func chatCmdFunc(command *cobra.Command, args []string) error {
    log := logger.New(app.Flags.Quiet)
    cfg, err := config.New(app)
    if err != nil {
        return err
    }
    srv, err := server.New(&cfg)
    if err != nil {
        return err
    }
    log.Print('Scan the following URL with a QR reader to join the chat session, press CTRL+C or "q" t

```

```

log.Print(srv.ChatJoinURL())
if err := qr.RenderString(srv.ChatJoinURL(), cfg.Reversed); err != nil {
    return err
}
if app.Flags.Browser {
    if err := srv.DisplayChat(); err != nil {
        return err
    }
} else {
    if err := srv.Chat(); err != nil {
        return err
    }
}
if err := keyboard.Open(); err == nil {
    defer func() {
        keyboard.Close()
    }()
    go func() {
        for {
            char, key, _ := keyboard.GetKey()
            if string(char) == "q" || key == keyboard.KeyCtrlC {
                srv.Shutdown()
            }
        }
    }()
} else {
    log.Print(fmt.Sprintf("Warning: keyboard not detected: %v", err))
}
return srv.Wait()
}

var chatCmd = &cobra.Command{
    Use:     "chat",
    Short:   "Start a browser chat session with another device",
    Long:    "Start a browser chat session where this host and a scanned mobile device can exchange tex",
    Aliases: []string{"c"},
    Example: `# Start a chat session and print a QR code
eqrcp chat
# Start a chat session and open the desktop browser interface
eqrcp chat --browser`,
    Args: cobra.NoArgs,
    RunE: chatCmdFunc,
}

package cmd

import (
    "fmt"

    "eqrcp/config"
    "github.com/spf13/cobra"

```

```

)

func configCmdFunc(command *cobra.Command, args []string) error {
    return config.Wizard(app)
}

var configCmd = &cobra.Command{
    Use:     "config",
    Short:   "Configure eqrcp",
    Long:    "Run an interactive configuration wizard for eqrcp. With this command you can configure wh
    Aliases: []string{"c", "cfg"},
    RunE:    configCmdFunc,
}

var migrateCmd = &cobra.Command{
    Use:     "migrate",
    Short:   "Migrate the legacy configuration file",
    Long:    "Migrate the legacy JSON configuration file to the new YAML format",
    Run: func(cmd *cobra.Command, args []string) {
        ok, err := config.Migrate(app)
        if err != nil {
            fmt.Println("error while migrating the legacy JSON configuration file:", err)
        }
        if ok {
            fmt.Println("Legacy JSON configuration file has been successfully deleted")
        }
    },
}
package cmd

import (
    "fmt"

    "eqrcp/version"
    "github.com/spf13/cobra"
)

var versionCmd = &cobra.Command{
    Use:     "version",
    Short:   "Print version number and build information.",
    Run: func(c *cobra.Command, args []string) {
        fmt.Println(version.String())
    },
}
package cmd

import (
    "os"

    "github.com/spf13/cobra"

```

```

)

// completionCmd represents the completion command
var completionCmd = &cobra.Command{
    Use:   "completion [bash|zsh|fish|powershell]",
    Short: "Generate completion script",
    Long:  "To load completions:

Bash:

$ source <(eqrcp completion bash)

# To load completions for each session, execute once:
Linux:
  $ eqrcp completion bash > /etc/bash_completion.d/eqrcp
MacOS:
  $ eqrcp completion bash > /usr/local/etc/bash_completion.d/eqrcp

Zsh:

# If shell completion is not already enabled in your environment you will need
# to enable it. You can execute the following once:

$ echo "autoload -U compinit; compinit" >> ~/.zshrc

# To load completions for each session, execute once:
$ eqrcp completion zsh > "${fpath[1]}/_eqrcp"

# You will need to start a new shell for this setup to take effect.

Fish:

$ eqrcp completion fish | source

# To load completions for each session, execute once:
$ eqrcp completion fish > ~/.config/fish/completions/eqrcp.fish
',
    DisableFlagsInUseLine: true,
    ValidArgs:             []string{"bash", "zsh", "fish", "powershell"},
    Args:                  cobra.MatchAll(cobra.ExactArgs(1), cobra.OnlyValidArgs),
    RunE: func(cmd *cobra.Command, args []string) error {
        switch args[0] {
        case "bash":
            if err := cmd.Root().GenBashCompletion(os.Stdout); err != nil {
                return err
            }
        case "zsh":
            if err := cmd.Root().GenZshCompletion(os.Stdout); err != nil {
                return err
            }
        }
    }
}

```

```

    case "fish":
        if err := cmd.Root().GenFishCompletion(os.Stdout, true); err != nil {
            return err
        }
    case "powershell":
        if err := cmd.Root().GenPowerShellCompletion(os.Stdout); err != nil {
            return err
        }
    }
    return nil
},
}
package cmd

import (
    "fmt"

    "eqrcp/application"
    "eqrcp/config"
    "eqrcp/version"
    "github.com/spf13/cobra"
)

var desktopCmd = &cobra.Command{
    Use:   "desktop",
    Short: "Desktop integration helpers",
    Long:  "Desktop integration helpers for file manager context menus and other non-terminal launchers"
}

var desktopShareCmd = &cobra.Command{
    Use:   "share <file-or-directory> [file-or-directory...]",
    Short: "Share selected files or directories from a desktop launcher",
    Long:  "Share selected files or directories from a desktop launcher. This command opens the QR code"
    Example: `# Share one file from a desktop launcher
eqrcp desktop share /path/file.txt
# Share multiple selected paths
eqrcp desktop share /path/file.txt /path/photo.jpg
# Share a directory
eqrcp desktop share /path/directory`,
    Args: cobra.MinimumNArgs(1),
    RunE: func(command *cobra.Command, args []string) error {
        app.Flags.Browser = desktopBrowserPreference(app.Flags, true)
        return sendCmdFunc(command, args)
    },
}

var desktopReceiveCmd = &cobra.Command{
    Use:   "receive [directory]",
    Short: "Receive files into a directory from a desktop launcher",
    Long:  "Receive files into a directory from a desktop launcher. This command opens the QR code in a

```

```

Example: '# Receive files into a selected directory
eqrcp desktop receive /path/directory
# Receive files into the configured output directory or current directory
eqrcp desktop receive',
Args: cobra.RangeArgs(0, 1),
RunE: func(command *cobra.Command, args []string) error {
    app.Flags.Browser = desktopBrowserPreference(app.Flags, true)
    if output, ok := desktopReceiveOutput(args); ok {
        app.Flags.Output = output
    } else if app.Flags.Output == "" {
        app.Flags.Output = desktopOutputPreference(app.Flags)
    }
    return receiveCmdFunc(command, args)
},
}

var desktopChatCmd = &cobra.Command{
    Use: "chat",
    Short: "Start a chat session from a desktop launcher",
    Long: "Start a browser-based chat session from a desktop launcher. This command opens the chat int
Example: '# Start a chat session
eqrcp desktop chat',
Args: cobra.NoArgs,
RunE: func(command *cobra.Command, args []string) error {
    app.Flags.Browser = desktopBrowserPreference(app.Flags, true)
    return chatCmdFunc(command, args)
},
}

func desktopReceiveOutput(args []string) (string, bool) {
    if len(args) == 0 {
        return "", false
    }
    return args[0], true
}

func desktopBrowserPreference(flags application.Flags, fallback bool) bool {
    settingsApp := application.New()
    settingsApp.Flags = flags
    settings, err := config.ReadDesktopSettings(settingsApp)
    if err != nil {
        return fallback
    }
    return settings.Browser
}

func desktopOutputPreference(flags application.Flags) string {
    settingsApp := application.New()
    settingsApp.Flags = flags
    settings, err := config.ReadDesktopSettings(settingsApp)

```

```

if err != nil || settings.Output == "" {
    return config.DefaultDesktopOutputDirectory()
}
return settings.Output
}

var desktopInstallCmd = &cobra.Command{
    Use: "install",
    Short: "Install desktop context menu entries",
    Long: "Install desktop context menu entries for the current user.",
    RunE: func(command *cobra.Command, args []string) error {
        if err := installDesktopIntegration(); err != nil {
            return err
        }
        fmt.Fprintln(command.OutOrStdout(), "Desktop context menu entries installed.")
        return nil
    },
}

var desktopStatusCmd = &cobra.Command{
    Use: "status",
    Short: "Show desktop context menu integration status",
    Long: "Show desktop context menu integration status for the current user.",
    RunE: func(command *cobra.Command, args []string) error {
        status, err := desktopIntegrationStatus()
        if err != nil {
            return err
        }
        fmt.Fprintf(command.OutOrStdout(), "%s\n%s", version.String(), status)
        return nil
    },
}

var desktopStartupEnableCmd = &cobra.Command{
    Use: "startup-enable",
    Short: "Start the desktop agent at login",
    Long: "Register the desktop integration agent to start when the current user logs in.",
    RunE: func(command *cobra.Command, args []string) error {
        if err := installDesktopStartup(); err != nil {
            return err
        }
        fmt.Fprintln(command.OutOrStdout(), "Desktop agent startup enabled.")
        return nil
    },
}

var desktopStartupDisableCmd = &cobra.Command{
    Use: "startup-disable",
    Short: "Stop starting the desktop agent at login",
    Long: "Remove the current-user login startup registration for the desktop integration agent.",

```

```

RunE: func(command *cobra.Command, args []string) error {
    if err := uninstallDesktopStartup(); err != nil {
        return err
    }
    fmt.Fprintln(command.OutOrStdout(), "Desktop agent startup disabled.")
    return nil
},
}

var desktopStartupStatusCmd = &cobra.Command{
    Use: "startup-status",
    Short: "Show desktop agent startup status",
    Long: "Show whether the desktop integration agent is registered to start when the current user log
RunE: func(command *cobra.Command, args []string) error {
    status, err := desktopStartupStatus()
    if err != nil {
        return err
    }
    fmt.Fprintf(command.OutOrStdout(), "%s\n%s", version.String(), status)
    return nil
},
}

var desktopUninstallCmd = &cobra.Command{
    Use: "uninstall",
    Short: "Uninstall desktop context menu entries",
    Long: "Uninstall desktop context menu entries created by eqrcp.",
    RunE: func(command *cobra.Command, args []string) error {
        if err := uninstallDesktopIntegration(); err != nil {
            return err
        }
        fmt.Fprintln(command.OutOrStdout(), "Desktop context menu entries uninstalled.")
        return nil
    },
}

package cmd

import (
    "context"
    "encoding/json"
    "fmt"
    "html/template"
    "io"
    "net"
    "net/http"
    "net/url"
    "os"
    "os/exec"
    "path/filepath"
    "runtime"

```

```

"strconv"
"strings"
"sync"
"time"

"eqrcp/application"
"eqrcp/body"
"eqrcp/config"
"eqrcp/server"
"eqrcp/version"
"github.com/adrg/xdg"
"github.com/spf13/cobra"
)

const desktopAgentAddress = "127.0.0.1:48176"
const desktopAgentMaxQueue = 16
const desktopAgentMaxHistory = 20
const desktopAgentHistoryFilename = "desktop-agent-history.json"

var openDesktopAgentPage = openDesktopAgentPageURL
var desktopAgentBaseURL = "http://" + desktopAgentAddress
var desktopAgentExecutable = os.Executable
var desktopAgentBackgroundStarter = startDesktopAgentBackgroundProcess
var desktopAgentReadyWaiter = waitForDesktopAgentReady

type desktopAgentTask struct {
    Action string    'json:"action"'
    Paths  []string 'json:"paths"'
}

type desktopAgentTaskRecord struct {
    ID                int        'json:"id"'
    Action            string     'json:"action"'
    Paths             []string  'json:"paths"'
    State            string     'json:"state"'
    TransferState     string     'json:"transferState,omitempty"'
    TransferMessage   string     'json:"transferMessage,omitempty"'
    TransferMode      string     'json:"transferMode,omitempty"'
    TransferTarget    string     'json:"transferTarget,omitempty"'
    TransferArchive   bool       'json:"transferArchive,omitempty"'
    TransferArchiveName string     'json:"transferArchiveName,omitempty"'
    TransferItems     []string  'json:"transferItems,omitempty"'
    TransferCurrent   string     'json:"transferCurrent,omitempty"'
    TransferPercent   int        'json:"transferPercent,omitempty"'
    BytesDone         int64     'json:"bytesDone,omitempty"'
    BytesTotal        int64     'json:"bytesTotal,omitempty"'
    SavedFiles        []string  'json:"savedFiles,omitempty"'
    ChatState         string     'json:"chatState,omitempty"'
    ChatMessageCount int        'json:"chatMessageCount,omitempty"'
    ChatDeviceCount  int        'json:"chatDeviceCount,omitempty"'

```

```

ChatLastActivity    string    'json:"chatLastActivity,omitempty"'
PageURL             string    'json:"pageUrl,omitempty"'
Error               string    'json:"error,omitempty"'
StartedAt           time.Time 'json:"startedAt"'
FinishedAt          *time.Time 'json:"finishedAt,omitempty"'
}

type desktopAgentResponse struct {
    State      string    'json:"state"'
    Current    *desktopAgentTaskRecord 'json:"current,omitempty"'
    Queued     int       'json:"queued"'
    History    []desktopAgentTaskRecord 'json:"history,omitempty"'
    LastError  string    'json:"lastError,omitempty"'
    Version    string    'json:"version"'
    AgentStartedAt time.Time 'json:"agentStartedAt"'
}

type desktopAgentHistoryStore struct {
    History []desktopAgentTaskRecord 'json:"history"'
}

type desktopAgentNotifier func(title string, message string) error

type desktopAgent struct {
    mu          sync.Mutex
    baseFlags   application.Flags
    startedAt   time.Time
    busy        bool
    current     *desktopAgentTaskRecord
    queue       []desktopAgentTask
    history     []desktopAgentTaskRecord
    nextID      int
    activeStop  func(string)
    shutdown    func()
    lastError   string
    runner      func(desktopAgentTask) error
    notifier    desktopAgentNotifier
    historyPath string
    notified    map[int]map[string]bool
    revision    int64
    subscribers map[chan struct{}]struct{}
}

func newDesktopAgent(baseFlags application.Flags) *desktopAgent {
    agent := &desktopAgent{
        baseFlags:   baseFlags,
        startedAt:   time.Now(),
        notifier:    notifyDesktop,
        historyPath: defaultDesktopAgentHistoryPath(),
        notified:    map[int]map[string]bool{},
    }

```

```

    subscribers: map[chan struct{}]struct{}{},
}
agent.runner = agent.runTask
return agent
}

func defaultDesktopAgentHistoryPath() string {
    return filepath.Join(xdg.ConfigHome, application.New().Name, desktopAgentHistoryFilename)
}

func (agent *desktopAgent) routes() http.Handler {
    mux := http.NewServeMux()
    mux.HandleFunc("/", agent.handlePage)
    mux.HandleFunc("/health", agent.handleHealth)
    mux.HandleFunc("/status", agent.handleStatus)
    mux.HandleFunc("/events", agent.handleEvents)
    mux.HandleFunc("/settings", agent.handleSettings)
    mux.HandleFunc("/restart", agent.handleRestart)
    mux.HandleFunc("/tasks", agent.handleTasks)
    mux.HandleFunc("/tasks/", agent.handleTaskAction)
    mux.HandleFunc("/history", agent.handleHistory)
    mux.HandleFunc("/stop-current", agent.handleStopCurrent)
    mux.HandleFunc("/shutdown", agent.handleShutdown)
    return mux
}

func (agent *desktopAgent) handlePage(w http.ResponseWriter, r *http.Request) {
    if r.URL.Path != "/" {
        http.NotFound(w, r)
        return
    }
    if r.Method != http.MethodGet {
        http.Error(w, "method not allowed", http.StatusMethodNotAllowed)
        return
    }
    status := agent.snapshot()
    w.Header().Set("Content-Type", "text/html; charset=utf-8")
    _ = desktopAgentPageTemplate.Execute(w, status)
}

func (agent *desktopAgent) handleHealth(w http.ResponseWriter, r *http.Request) {
    if handleDesktopAgentCORS(w, r, http.MethodGet) {
        w.WriteHeader(http.StatusNoContent)
        return
    }
    if rejectCrossOriginDesktopAgent(w, r) {
        return
    }
    if r.Method != http.MethodGet {
        http.Error(w, "method not allowed", http.StatusMethodNotAllowed)
    }
}

```

```

    return
}
w.WriteHeader(http.StatusNoContent)
}

func (agent *desktopAgent) handleStatus(w http.ResponseWriter, r *http.Request) {
    if handleDesktopAgentCORS(w, r, http.MethodGet) {
        w.WriteHeader(http.StatusNoContent)
        return
    }
    if rejectCrossOriginDesktopAgent(w, r) {
        return
    }
    if r.Method != http.MethodGet {
        http.Error(w, "method not allowed", http.StatusMethodNotAllowed)
        return
    }
    agent.writeStatus(w)
}

func (agent *desktopAgent) handleEvents(w http.ResponseWriter, r *http.Request) {
    if handleDesktopAgentCORS(w, r, http.MethodGet) {
        w.WriteHeader(http.StatusNoContent)
        return
    }
    if rejectCrossOriginDesktopAgent(w, r) {
        return
    }
    if r.Method != http.MethodGet {
        http.Error(w, "method not allowed", http.StatusMethodNotAllowed)
        return
    }
    flusher, ok := w.(http.Flusher)
    if !ok {
        http.Error(w, "streaming unsupported", http.StatusInternalServerError)
        return
    }
    w.Header().Set("Content-Type", "text/event-stream")
    w.Header().Set("Cache-Control", "no-cache")
    w.Header().Set("Connection", "keep-alive")
    lastRevision := int64(-1)
    send := func() bool {
        status, revision := agent.snapshotWithRevision()
        if revision == lastRevision {
            return true
        }
        lastRevision = revision
        if _, err := fmt.Fprint(w, "data: "); err != nil {
            return false
        }
    }

```

```

    if err := json.NewEncoder(w).Encode(status); err != nil {
        return false
    }
    if _, err := fmt.Fprint(w, "\n"); err != nil {
        return false
    }
    flusher.Flush()
    return true
}
if !send() {
    return
}
events, unsubscribe := agent.subscribeEvents()
defer unsubscribe()
heartbeat := time.NewTicker(15 * time.Second)
defer heartbeat.Stop()
for {
    select {
    case <-r.Context().Done():
        return
    case <-events:
        if !send() {
            return
        }
    case <-heartbeat.C:
        if _, err := fmt.Fprint(w, ": keep-alive\n\n"); err != nil {
            return
        }
        flusher.Flush()
    }
}
}
}

func (agent *desktopAgent) subscribeEvents() (<-chan struct{}, func()) {
    ch := make(chan struct{}, 1)
    agent.mu.Lock()
    if agent.subscribers == nil {
        agent.subscribers = map[chan struct{}]struct{}{}
    }
    agent.subscribers[ch] = struct{}{}
    agent.mu.Unlock()
    return ch, func() {
        agent.mu.Lock()
        delete(agent.subscribers, ch)
        close(ch)
        agent.mu.Unlock()
    }
}

func (agent *desktopAgent) touchLocked() {

```

```

agent.revision++
for ch := range agent.subscribers {
    select {
        case ch <- struct{}{}:
        default:
        }
    }
}

func (agent *desktopAgent) handleRestart(w http.ResponseWriter, r *http.Request) {
    if rejectCrossOriginDesktopAgent(w, r) {
        return
    }
    if r.Method != http.MethodPost {
        http.Error(w, "method not allowed", http.StatusMethodNotAllowed)
        return
    }
    logFile, logPath, err := createDesktopAgentBackgroundLog()
    if err != nil {
        http.Error(w, fmt.Sprintf("desktop agent restart failed: %v", err), http.StatusInternalServerError)
        return
    }
    exe, err := desktopAgentExecutable()
    if err != nil {
        _ = logFile.Close()
        http.Error(w, fmt.Sprintf("desktop agent restart failed: %v", err), http.StatusInternalServerError)
        return
    }
    cmd := exec.Command(exe, "desktop", "agent-restart-helper")
    configureDesktopAgentBackgroundCommand(cmd)
    cmd.Stdout = logFile
    cmd.Stderr = logFile
    if err := cmd.Start(); err != nil {
        _ = logFile.Close()
        http.Error(w, fmt.Sprintf("desktop agent restart failed: %v", err), http.StatusInternalServerError)
        return
    }
    if err := cmd.Process.Release(); err != nil {
        _ = logFile.Close()
        http.Error(w, fmt.Sprintf("desktop agent restart failed: %v", err), http.StatusInternalServerError)
        return
    }
    agent.mu.Lock()
    agent.queue = nil
    if agent.busy {
        agent.finalizeActiveLocked("replaced")
    }
    shutdown := agent.shutdown
    agent.touchLocked()
    agent.mu.Unlock()
}

```

```

w.WriteHeader(http.StatusAccepted)
fmt.Fprintf(w, "Desktop agent restarting.\nLog: %s\n", logPath)
go func() {
    defer logFile.Close()
    if shutdown != nil {
        shutdown()
    }
}()
}

func (agent *desktopAgent) handleSettings(w http.ResponseWriter, r *http.Request) {
    if rejectCrossOriginDesktopAgent(w, r) {
        return
    }
    switch r.Method {
    case http.MethodGet:
        settings, err := agent.readSettings()
        if err != nil {
            http.Error(w, fmt.Sprintf("desktop agent settings unavailable: %v", err), http.StatusInternalServerError)
            return
        }
        w.Header().Set("Content-Type", "application/json")
        _ = json.NewEncoder(w).Encode(settings)
    case http.MethodPost:
        var settings config.DesktopSettings
        if err := json.NewDecoder(r.Body).Decode(&settings); err != nil {
            http.Error(w, fmt.Sprintf("invalid settings: %v", err), http.StatusBadRequest)
            return
        }
        saved, err := agent.writeSettings(settings)
        if err != nil {
            http.Error(w, fmt.Sprintf("save desktop agent settings failed: %v", err), http.StatusBadRequest)
            return
        }
        w.Header().Set("Content-Type", "application/json")
        _ = json.NewEncoder(w).Encode(saved)
    default:
        http.Error(w, "method not allowed", http.StatusMethodNotAllowed)
    }
}

func (agent *desktopAgent) settingsApp() application.App {
    settingsApp := application.New()
    settingsApp.Flags = agent.baseFlags
    return settingsApp
}

func (agent *desktopAgent) readSettings() (config.DesktopSettings, error) {
    return config.ReadDesktopSettings(agent.settingsApp())
}

```

```

}

func (agent *desktopAgent) writeSettings(settings config.DesktopSettings) (config.DesktopSettin
return config.WriteDesktopSettings(agent.settingsApp(), settings)
}

func (agent *desktopAgent) handleTasks(w http.ResponseWriter, r *http.Request) {
    if rejectCrossOriginDesktopAgent(w, r) {
        return
    }
    if r.Method != http.MethodPost {
        http.Error(w, "method not allowed", http.StatusMethodNotAllowed)
        return
    }
    var task desktopAgentTask
    if err := json.NewDecoder(r.Body).Decode(&task); err != nil {
        http.Error(w, fmt.Sprintf("invalid task: %v", err), http.StatusBadRequest)
        return
    }
    if err := validateDesktopAgentTask(task); err != nil {
        http.Error(w, err.Error(), http.StatusBadRequest)
        return
    }
    agent.mu.Lock()
    if len(agent.queue) >= desktopAgentMaxQueue {
        agent.mu.Unlock()
        http.Error(w, "desktop agent queue is full", http.StatusTooManyRequests)
        return
    }
    agent.queue = append(agent.queue, task)
    agent.lastError = ""
    if agent.busy {
        agent.replaceActiveLocked("replaced")
    }
    agent.startNextLocked()
    agent.touchLocked()
    agent.mu.Unlock()

    w.WriteHeader(http.StatusAccepted)
    agent.writeStatus(w)
}

func (agent *desktopAgent) handleTaskAction(w http.ResponseWriter, r *http.Request) {
    if handleDesktopAgentCORS(w, r, http.MethodPost) {
        w.WriteHeader(http.StatusNoContent)
        return
    }
    if rejectCrossOriginDesktopAgent(w, r) {
        return
    }
}

```

```

    if r.Method != http.MethodPost {
        http.Error(w, "method not allowed", http.StatusMethodNotAllowed)
        return
    }
    taskID, action, ok := parseDesktopAgentTaskActionPath(r.URL.Path)
    if !ok || action != "repeat" {
        http.NotFound(w, r)
        return
    }
    if err := agent.repeatTask(taskID); err != nil {
        http.Error(w, err.Error(), http.StatusConflict)
        return
    }
    w.WriteHeader(http.StatusAccepted)
    agent.writeStatus(w)
}

func parseDesktopAgentTaskActionPath(path string) (int, string, bool) {
    parts := strings.Split(strings.Trim(path, "/"), "/")
    if len(parts) != 3 || parts[0] != "tasks" {
        return 0, "", false
    }
    id, err := strconv.Atoi(parts[1])
    if err != nil || id <= 0 {
        return 0, "", false
    }
    return id, parts[2], true
}

func handleDesktopAgentCORS(w http.ResponseWriter, r *http.Request, methods ...string) bool {
    origin := r.Header.Get("Origin")
    if origin != "" && trustedDesktopAgentOrigin(origin, r.Host) {
        w.Header().Set("Access-Control-Allow-Origin", origin)
        w.Header().Set("Vary", "Origin")
        w.Header().Set("Access-Control-Allow-Headers", "Content-Type")
        if len(methods) > 0 {
            allowed := append([]string(nil), methods...)
            allowed = append(allowed, http.MethodOptions)
            w.Header().Set("Access-Control-Allow-Methods", strings.Join(allowed, ", "))
        }
    }
    return r.Method == http.MethodOptions
}

func rejectCrossOriginDesktopAgent(w http.ResponseWriter, r *http.Request) bool {
    origin := r.Header.Get("Origin")
    if origin == "" {
        return false
    }
    if trustedDesktopAgentOrigin(origin, r.Host) {

```

```

    return false
}
http.Error(w, "forbidden", http.StatusForbidden)
return true
}

func trustedDesktopAgentOrigin(origin string, requestHost string) bool {
    parsed, err := url.Parse(origin)
    if err != nil {
        return false
    }
    if trustedDesktopAgentWailsOrigin(parsed) {
        return true
    }
    if strings.EqualFold(parsed.Host, requestHost) {
        return true
    }
    return trustedDesktopAgentLocalHost(parsed.Hostname())
}

func trustedDesktopAgentWailsOrigin(parsed *url.URL) bool {
    host := strings.ToLower(parsed.Hostname())
    return parsed.Scheme == "wails" || host == "wails.localhost"
}

func trustedDesktopAgentLocalHost(host string) bool {
    host = strings.Trim(strings.ToLower(host), "[ ]")
    if host == "localhost" {
        return true
    }
    ip := net.ParseIP(host)
    return ip != nil && (ip.IsLoopback() || ip.IsPrivate() || ip.IsLinkLocalUnicast())
}

func (agent *desktopAgent) handleHistory(w http.ResponseWriter, r *http.Request) {
    if rejectCrossOriginDesktopAgent(w, r) {
        return
    }
    if r.Method != http.MethodDelete {
        http.Error(w, "method not allowed", http.StatusMethodNotAllowed)
        return
    }
    if err := agent.clearHistory(); err != nil {
        http.Error(w, fmt.Sprintf("clear desktop agent history failed: %v", err), http.StatusInternalServerError)
        return
    }
    w.WriteHeader(http.StatusNoContent)
}

func (agent *desktopAgent) handleStopCurrent(w http.ResponseWriter, r *http.Request) {

```

```

    if rejectCrossOriginDesktopAgent(w, r) {
        return
    }
    if r.Method != http.MethodPost {
        http.Error(w, "method not allowed", http.StatusMethodNotAllowed)
        return
    }
    if !agent.stopCurrent("stopped") {
        http.Error(w, "desktop agent has no active task", http.StatusConflict)
        return
    }
    if strings.Contains(r.Header.Get("Accept"), "text/html") {
        http.Redirect(w, r, "/", http.StatusSeeOther)
        return
    }
    w.WriteHeader(http.StatusAccepted)
    fmt.Fprintln(w, "Current desktop agent task stopped.")
}

func (agent *desktopAgent) handleShutdown(w http.ResponseWriter, r *http.Request) {
    if rejectCrossOriginDesktopAgent(w, r) {
        return
    }
    if r.Method != http.MethodPost {
        http.Error(w, "method not allowed", http.StatusMethodNotAllowed)
        return
    }
    agent.mu.Lock()
    agent.queue = nil
    if agent.busy {
        agent.finalizeActiveLocked("replaced")
    }
    shutdown := agent.shutdown
    agent.touchLocked()
    agent.mu.Unlock()

    w.WriteHeader(http.StatusAccepted)
    fmt.Fprintln(w, "Desktop agent stopping.")
    if shutdown != nil {
        go shutdown()
    }
}

func (agent *desktopAgent) execute(task desktopAgentTask, id int) {
    err := agent.runner(task)
    agent.mu.Lock()
    defer agent.mu.Unlock()
    if agent.current != nil && agent.current.ID == id {
        finishedAt := time.Now()
        agent.current.FinishedAt = &finishedAt
    }
}

```

```

if agent.current.State == "running" {
    if err != nil {
        agent.current.State = "failed"
        agent.current.Error = err.Error()
        agent.lastError = err.Error()
    } else if isTerminalDesktopTransferState(agent.current.TransferState) {
        agent.current.State = agent.current.TransferState
    } else {
        agent.current.State = "completed"
    }
}
agent.addHistoryLocked(*agent.current)
agent.notifyRecordLocked(*agent.current)
delete(agent.notified, agent.current.ID)
agent.busy = false
agent.current = nil
agent.activeStop = nil
agent.startNextLocked()
agent.touchLocked()
}
}

func (agent *desktopAgent) stopCurrent(state string) bool {
    agent.mu.Lock()
    defer agent.mu.Unlock()
    if !agent.busy {
        return false
    }
    agent.replaceActiveLocked(state)
    agent.touchLocked()
    return true
}

func (agent *desktopAgent) repeatTask(id int) error {
    agent.mu.Lock()
    defer agent.mu.Unlock()
    if len(agent.queue) >= desktopAgentMaxQueue {
        return fmt.Errorf("desktop agent queue is full")
    }
    var repeated *desktopAgentTask
    if agent.current != nil && agent.current.ID == id {
        task := desktopAgentTask{Action: agent.current.Action, Paths: append([]string(nil), agent.curre
        repeated = &task
    }
    if repeated == nil {
        for _, record := range agent.history {
            if record.ID == id {

```

```

    }
  }
}
if repeated == nil {
    return fmt.Errorf("desktop agent task #%d was not found", id)
}
if err := validateDesktopAgentTask(*repeated); err != nil {
    return err
}
agent.queue = append(agent.queue, *repeated)
agent.lastError = ""
if agent.busy {
    agent.replaceActiveLocked("replaced")
}
agent.startNextLocked()
agent.touchLocked()
return nil
}

func isTerminalDesktopTransferState(state string) bool {
    return state == "completed" || state == "stopped" || state == "failed"
}

func isTerminalDesktopChatState(state string) bool {
    return state == "ended" || state == "stopped" || state == "failed" || state == "replaced"
}

func desktopTaskStateForChatState(state string) string {
    switch state {
    case "ended":
        return "completed"
    case "stopped", "failed", "replaced":
        return state
    default:
        return "running"
    }
}

func (agent *desktopAgent) replaceActiveLocked(state string) {
    if agent.current != nil && agent.current.State == "running" {
        agent.current.State = state
        finishedAt := time.Now()
        agent.current.FinishedAt = &finishedAt
    }
    if agent.activeStop == nil {
        return
    }
    stop := agent.activeStop
    go stop(state)
}

```

```

func (agent *desktopAgent) finalizeActiveLocked(state string) {
    agent.replaceActiveLocked(state)
    if agent.current == nil {
        return
    }
    record := *agent.current
    agent.addHistoryLocked(record)
    agent.notifyRecordLocked(record)
    delete(agent.notified, record.ID)
    agent.busy = false
    agent.current = nil
    agent.activeStop = nil
}

func (agent *desktopAgent) startNextLocked() {
    if agent.busy || len(agent.queue) == 0 {
        return
    }
    task := agent.queue[0]
    agent.queue = agent.queue[1:]
    agent.nextID++
    record := desktopAgentTaskRecord{
        ID:      agent.nextID,
        Action:   task.Action,
        Paths:    append([]string(nil), task.Paths...),
        State:    "running",
        StartedAt: time.Now(),
    }
    agent.busy = true
    agent.current = &record
    agent.notifyRecordLocked(record)
    go agent.execute(task, record.ID)
}

func (agent *desktopAgent) currentTaskID() int {
    agent.mu.Lock()
    defer agent.mu.Unlock()
    if agent.current == nil {
        return 0
    }
    return agent.current.ID
}

func (agent *desktopAgent) observeTransferStatus(taskID int, status server.TransferStatusSnapshot) {
    agent.mu.Lock()
    defer agent.mu.Unlock()
    if agent.current == nil || agent.current.ID != taskID {
        return
    }
}

```

```

agent.current.TransferState = status.State
agent.current.TransferMessage = status.Message
agent.current.TransferMode = status.Mode
agent.current.TransferTarget = status.Target
agent.current.TransferArchive = status.Archive
agent.current.TransferArchiveName = status.ArchiveName
agent.current.TransferItems = append([]string(nil), status.Items...)
agent.current.TransferCurrent = status.Current
agent.current.TransferPercent = status.Percent
agent.current.BytesDone = status.BytesDone
agent.current.BytesTotal = status.BytesTotal
agent.current.SavedFiles = append([]string(nil), status.SavedFiles...)
if isTerminalDesktopTransferState(status.State) && agent.current.State == "running" {
    agent.current.State = status.State
    finishedAt := time.Now()
    agent.current.FinishedAt = &finishedAt
}
agent.notifyTransferStatusLocked(*agent.current)
if isTerminalDesktopTransferState(agent.current.State) {
    record := *agent.current
    agent.addHistoryLocked(record)
    agent.notifyRecordLocked(record)
    delete(agent.notified, record.ID)
    agent.busy = false
    agent.current = nil
    agent.activeStop = nil
    agent.startNextLocked()
    agent.touchLocked()
    return
}
agent.touchLocked()
}

func (agent *desktopAgent) observeChatStatus(taskID int, status server.ChatStatusSnapshot) {
    agent.mu.Lock()
    defer agent.mu.Unlock()
    if agent.current == nil || agent.current.ID != taskID {
        return
    }
    agent.current.ChatState = status.State
    agent.current.ChatMessageCount = status.MessageCount
    agent.current.ChatDeviceCount = status.DeviceCount
    if !status.LastActivity.IsZero() {
        agent.current.ChatLastActivity = status.LastActivity.Format(time.RFC3339)
    }
    if isTerminalDesktopChatState(status.State) && agent.current.State == "running" {
        agent.current.State = desktopTaskStateForChatState(status.State)
        finishedAt := time.Now()
        agent.current.FinishedAt = &finishedAt
        record := *agent.current
    }
}

```

```

    agent.addHistoryLocked(record)
    agent.notifyRecordLocked(record)
    delete(agent.notified, record.ID)
    agent.busy = false
    agent.current = nil
    agent.activeStop = nil
    agent.startNextLocked()
    agent.touchLocked()
    return
}
agent.touchLocked()
}

func (agent *desktopAgent) notifyRecordLocked(record desktopAgentTaskRecord) {
    if agent.notifier == nil {
        return
    }
    title, message := desktopAgentNotification(record)
    if title == "" || message == "" {
        return
    }
    _ = agent.notifier(title, message)
}

func (agent *desktopAgent) notifyTransferStatusLocked(record desktopAgentTaskRecord) {
    if agent.notifier == nil {
        return
    }
    key := record.TransferState
    switch key {
    case "transferring", "completed", "stopped", "failed":
    default:
        return
    }
    if agent.notified[record.ID] == nil {
        agent.notified[record.ID] = map[string]bool{}
    }
    if agent.notified[record.ID][key] {
        return
    }
    title, message := desktopAgentTransferNotification(record)
    if title == "" || message == "" {
        return
    }
    agent.notified[record.ID][key] = true
    _ = agent.notifier(title, message)
}

func desktopAgentNotification(record desktopAgentTaskRecord) (string, string) {
    if record.Action == "chat" {

```

```

    return "", ""
}
action := desktopAgentActionLabel(record.Action)
target := desktopAgentPathsSummary(record.Paths)
switch record.State {
case "running":
    return "eqrcp transfer ready", fmt.Sprintf("%s ready: %s", action, target)
case "completed":
    if record.TransferState == "completed" {
        return "", ""
    }
    return "eqrcp transfer completed", fmt.Sprintf("%s completed: %s", action, target)
case "failed":
    if record.TransferState == "failed" {
        return "", ""
    }
    if record.Error != "" {
        return "eqrcp transfer failed", fmt.Sprintf("%s failed: %s", action, record.Error)
    }
    return "eqrcp transfer failed", fmt.Sprintf("%s failed: %s", action, target)
case "stopped":
    if record.TransferState == "stopped" {
        return "", ""
    }
    return "eqrcp transfer stopped", fmt.Sprintf("%s stopped: %s", action, target)
case "replaced":
    return "eqrcp transfer replaced", fmt.Sprintf("%s replaced by a newer task: %s", action, target)
default:
    return "", ""
}
}

func desktopAgentTransferNotification(record desktopAgentTaskRecord) (string, string) {
    action := desktopAgentActionLabel(record.Action)
    target := desktopAgentPathsSummary(record.Paths)
    if record.TransferCurrent != "" {
        target = record.TransferCurrent
    }
    switch record.TransferState {
case "transferring":
    return "eqrcp transfer started", fmt.Sprintf("%s started: %s", action, target)
case "completed":
    if len(record.SavedFiles) == 1 {

```

```

tmpfile, err = os.OpenFile(tmpfile.Name()+".zip", os.O_RDWR, 0644)
if err != nil {
    return "", err
}
if err := zip.CreateWriter(tmpfile.Name(), tmpfile); err != nil {
    return "", err
}

for _, filename := range files {
    fileinfo, err := os.Stat(filename)
    if err != nil {
        return "", err
    }
    if fileinfo.IsDir() {
        if err := zip.AddAll(filename, true); err != nil {
            return "", err
        }
    } else {
        file, err := os.Open(filename)
        if err != nil {
            return "", err
        }
        addErr := zip.Add(filename, file, fileinfo)
        file.Close()
        if addErr != nil {
            return "", addErr
        }
    }
}

if err := zip.Writer.Close(); err != nil {
    return "", err
}
if err := tmpfile.Close(); err != nil {
    return "", err
}

return zip.Name, nil
}

// GetRandomURLPath returns an unguessable URL-safe path segment.
func GetRandomURLPath() (string, error) {
    randbytes := make([]byte, 18)
    if _, err := io.ReadFull(rand.Reader, randbytes); err != nil {
        return "", err
    }
    return base64.RawURLEncoding.EncodeToString(randbytes), nil
}

// GetSessionID returns a base64 encoded string of 40 random characters

```

```

func GetSessionID() (string, error) {
    randbytes := make([]byte, 40)
    if _, err := io.ReadFull(rand.Reader, randbytes); err != nil {
        return "", err
    }
    return base64.StdEncoding.EncodeToString(randbytes), nil
}

// GetInterfaceAddress returns the address of the network interface to
// bind the server to. If the interface is "any", it will return 0.0.0.0.
// If no interface is found with that name, an error is returned
func GetInterfaceAddress(ifaceString string) (string, error) {
    if ifaceString == "any" {
        return "0.0.0.0", nil
    }
    ifaces, err := net.Interfaces()
    if err != nil {
        return "", err
    }
    var candidateInterface *net.Interface
    for _, iface := range ifaces {
        if iface.Name == ifaceString {
            candidateInterface = &iface
            break
        }
    }
    if candidateInterface != nil {
        ip, err := FindIP(*candidateInterface)
        if err != nil {
            return "", err
        }
        return ip, nil
    }
    return "", errors.New("unable to find interface")
}

// FindIP returns the IP address of the passed interface, and an error
func FindIP(iface net.Interface) (string, error) {
    var ip string
    addrs, err := iface.Addrs()
    if err != nil {
        return "", err
    }
    for _, addr := range addrs {
        if ipnet, ok := addr.(*net.IPNet); ok {
            if ipnet.IP.IsLinkLocalUnicast() {
                continue
            }
            if ipnet.IP.To4() != nil {
                ip = ipnet.IP.String()
            }
        }
    }
    return ip, nil
}

```

```

        continue
    }
    // Use IPv6 only if an IPv4 hasn't been found yet.
    // This is eventually overwritten with an IPv4, if found (see above)
    if ip == "" {
        ip = "[" + ipnet.IP.String() + "]"
    }
}
}
if ip == "" {
    return "", errors.New("unable to find an IP for this interface")
}
return ip, nil
}

// ReadFileNames from dir
func ReadFileNames(dir string) ([]string, error) {
    files, err := os.ReadDir(dir)
    if err != nil {
        return nil, err
    }
    // Create array of names of files which are stored in dir
    // used later to set valid name for received files
    filenames := make([]string, 0, len(files))
    for _, fi := range files {
        filenames = append(filenames, fi.Name())
    }
    return filenames, nil
}

package util

import (
    "net"
    "regexp"

    externalip "github.com/glendc/go-external-ip"
)

// Interfaces returns a 'name:ip' map of the suitable interfaces found
func Interfaces(listAll bool) (map[string]string, error) {
    names := make(map[string]string)
    ifaces, err := net.Interfaces()
    if err != nil {
        return names, err
    }
}

```

```

    if iface.Flags&net.FlagUp == 0 {
        continue
    }
    ip, err := FindIP(iface)
    if err != nil {
        continue
    }
    names[iface.Name] = ip
}
return names, nil
}

// GetExternalIP of this host
func GetExternalIP() (net.IP, error) {
    consensus := externalip.DefaultConsensus(nil, nil)
    // Get your IP, which is never <nil> when err is <nil>
    ip, err := consensus.ExternalIP()
    if err != nil {
        return nil, err
    }
    return ip, nil
}
package logger

import (
    "fmt"
)

// Print prints its argument if the --quiet flag is not passed
func (l Logger) Print(args ...interface{}) {
    if !l.quiet {
        fmt.Println(args...)
    }
}

// Logger struct
type Logger struct {
    quiet bool
}

// New logger
func New(quiet bool) Logger {
    return Logger{
        quiet: quiet,
    }
}
package pages

import _ "embed"

```

```

// QR page
//
//go:embed qr.tmpl.html
var QR string

// Chat page
//
//go:embed chat.tmpl.html
var Chat string

// Upload page
//
//go:embed upload.tmpl.html
var Upload string

// Done page
//
//go:embed done.tmpl.html
var Done string
package version

import (
    "fmt"
    "os"
    "time"
)

var (
    app      = "eqrcp"
    version  = "dev"
    date     = "n/a"
)

// String returns a string representation of the build.
func String() string {
    date := buildDate()
    return fmt.Sprintf("%s %s [date: %s]", app, displayVersion(date), date)
}

func displayVersion(buildDate string) string {
    if version != "dev" {
        return version
    }
    token := compactBuildDate(buildDate)
    if token == "" {
        return version
    }
    return version + "-" + token
}

```

```

func compactBuildDate(value string) string {
    if value == "" || value == "n/a" {
        return ""
    }
    buffer := make([]byte, 0, len(value))
    for index := 0; index < len(value); index++ {
        ch := value[index]
        if (ch >= '0' && ch <= '9') || (ch >= 'A' && ch <= 'Z') || (ch >= 'a' && ch <= 'z') {
            buffer = append(buffer, ch)
        }
    }
    return string(buffer)
}

func buildDate() string {
    if date != "" && date != "n/a" {
        return date
    }
    exe, err := os.Executable()
    if err != nil {
        return date
    }
    info, err := os.Stat(exe)
    if err != nil {
        return date
    }
    return info.ModTime().Format(time.RFC3339Nano)
}

package main

import (
    "context"
    "embed"
    "net/http"

    "github.com/wailsapp/wails/v2"
    "github.com/wailsapp/wails/v2/pkg/options"
    "github.com/wailsapp/wails/v2/pkg/options/assetserver"
)

//go:embed all:frontend/dist
var assets embed.FS

func main() {
    // Create an instance of the app structure
    app := NewApp()
    tray := newTrayController(app)

    // Create application with options
    err := wails.Run(&options.App{

```

```

Title:           "EQT",
Width:           1120,
Height:          760,
MinWidth:        900,
MinHeight:       640,
HideWindowOnClose: false,
AssetServer: &assetserver.Options{
  Assets: assets,
  // Inject CSP that allows the chat iframe (served by the local agent
  // HTTP server at 127.0.0.1) to load inside this Wails webview.
  Middleware: func(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
      w.Header().Set("Content-Security-Policy",
        "default-src 'self' 'unsafe-inline' 'unsafe-eval'; "+
        "connect-src 'self' http://127.0.0.1:* http://localhost:*; "+
        "img-src 'self' data: http://127.0.0.1:* http://localhost:* http://*:* https://*:*; "+
        "frame-src http://*:* https://*:*")
      next.ServeHTTP(w, r)
    })
  },
},
BackgroundColour: &options.RGBA{R: 245, G: 247, B: 244, A: 1},
SingleInstanceLock: &options.SingleInstanceLock{
  UniqueId: "eqrcp-desktop",
  OnSecondInstanceLaunch: func(data options.SecondInstanceData) {
    app.showWindow()
  },
},
OnStartup: func(ctx context.Context) {
  app.startup(ctx)
  tray.startTray()
},
OnBeforeClose: app.beforeClose,
OnShutdown: func(ctx context.Context) {
  tray.shutdown()
},
DragAndDrop: &options.DragAndDrop{
  EnableFileDrop: true,
  DisableWebViewDrop: true,
},
Bind: []interface{}{
  app,
},
})

if err != nil {
  println("Error:", err.Error())
}

package main

```

```

import (
    "bytes"
    "context"
    "encoding/json"
    "fmt"
    "io"
    "net/http"
    "net/url"
    "os"
    "os/exec"
    "path/filepath"
    "runtime"
    "strings"
    "sync"
    "time"

    wailsruntime "github.com/wailsapp/wails/v2/pkg/runtime"
)

const agentBaseURL = "http://127.0.0.1:48176"
const chatSaveRetentionDays = 7

type App struct {
    ctx          context.Context
    client       *http.Client
    mu           sync.Mutex
    closeBehavior string
    forceQuit    bool
}

type AgentTask struct {
    Action string `json:"action"`
    Paths  []string `json:"paths"`
}

type TaskRecord struct {
    ID            int      `json:"id"`
    Action        string   `json:"action"`
    Paths         []string `json:"paths"`
    State         string   `json:"state"`
    TransferState string   `json:"transferState,omitempty"`
    TransferMessage string  `json:"transferMessage,omitempty"`
    TransferMode  string   `json:"transferMode,omitempty"`
    TransferTarget string  `json:"transferTarget,omitempty"`
    TransferArchiveName string `json:"transferArchiveName,omitempty"`
    TransferCurrent string  `json:"transferCurrent,omitempty"`
    TransferPercent int      `json:"transferPercent,omitempty"`
    BytesDone     int64   `json:"bytesDone,omitempty"`
    BytesTotal    int64   `json:"bytesTotal,omitempty"`
}

```

```

SavedFiles      []string  'json:"savedFiles,omitempty"'
ChatState       string    'json:"chatState,omitempty"'
ChatMessageCount int      'json:"chatMessageCount,omitempty"'
ChatDeviceCount int      'json:"chatDeviceCount,omitempty"'
ChatLastActivity string   'json:"chatLastActivity,omitempty"'
PageURL         string    'json:"pageUrl,omitempty"'
Error           string    'json:"error,omitempty"'
StartedAt       time.Time 'json:"startedAt"'
FinishedAt      *time.Time 'json:"finishedAt,omitempty"'
}

```

```

type AgentStatus struct {
    State      string    'json:"state"'
    Current    *TaskRecord 'json:"current,omitempty"'
    Queued     int      'json:"queued"'
    History    []TaskRecord 'json:"history,omitempty"'
    LastError  string    'json:"lastError,omitempty"'
    Version    string    'json:"version"'
    AgentStartedAt time.Time 'json:"agentStartedAt"'
}

```

```

type DesktopSettings struct {
    ConfigPath    string    'json:"configPath"'
    Interface     string    'json:"interface"'
    InterfaceOptions []InterfaceOption 'json:"interfaceOptions"'
    Port         int      'json:"port"'
    Output       string    'json:"output"'
    Browser      bool     'json:"browser"'
    ChatAutoSave bool     'json:"chatAutoSave"'
    CloseBehavior string    'json:"closeBehavior"'
    ChatSender   string    'json:"chatSender"'
    ChatAvatar   string    'json:"chatAvatar"'
}

```

```

type InterfaceOption struct {
    Name string 'json:"name"'
    IP   string 'json:"ip"'
    Label string 'json:"label"'
}

```

```

type AppInfo struct {
    Product    string 'json:"product"'
    Name       string 'json:"name"'
    Description string 'json:"description"'
    AgentURL   string 'json:"agentUrl"'
    OS        string 'json:"os"'
    Arch       string 'json:"arch"'
    CLIPath    string 'json:"cliPath,omitempty"'
}

```

```

func NewApp() *App {
    return &App{
        client:      &http.Client{Timeout: 5 * time.Second},
        closeBehavior: "tray",
    }
}

func (a *App) startup(ctx context.Context) {
    a.ctx = ctx
}

func (a *App) showWindow() {
    if a.ctx == nil {
        return
    }
    wailsruntime.WindowUnminimise(a.ctx)
    wailsruntime.WindowShow(a.ctx)
}

func (a *App) beforeClose(ctx context.Context) bool {
    if a.consumeForceQuit() || a.currentCloseBehavior() == "quit" {
        return false
    }
    wailsruntime.WindowHide(ctx)
    return true
}

func (a *App) quit() {
    a.mu.Lock()
    a.forceQuit = true
    a.mu.Unlock()
    if a.ctx != nil {
        wailsruntime.Quit(a.ctx)
    }
}

func (a *App) consumeForceQuit() bool {
    a.mu.Lock()
    defer a.mu.Unlock()
    if !a.forceQuit {
        return false
    }
    a.forceQuit = false
    return true
}

func (a *App) currentCloseBehavior() string {
    a.mu.Lock()
    defer a.mu.Unlock()
    return a.closeBehavior
}

```

```

}

func (a *App) setCloseBehavior(value string) {
    if value != "quit" {
        value = "tray"
    }
    a.mu.Lock()
    a.closeBehavior = value
    a.mu.Unlock()
}

func (a *App) emitTrayCommand(command string) {
    if a.ctx == nil {
        return
    }
    wailsruntime.EventsEmit(a.ctx, "eqt:tray-command", command)
}

func (a *App) AgentStatus() (AgentStatus, error) {
    if err := a.ensureAgent(); err != nil {
        return AgentStatus{}, err
    }
    var status AgentStatus
    if err := a.getJSON("/status", &status); err != nil {
        return AgentStatus{}, err
    }
    return status, nil
}

func (a *App) Share(paths []string) (AgentStatus, error) {
    if len(paths) == 0 {
        return AgentStatus{}, fmt.Errorf("choose at least one file or folder")
    }
    return a.postTask(AgentTask{Action: "share", Paths: paths})
}

func (a *App) Receive(output string) (AgentStatus, error) {
    paths := []string{}
    if output != "" {
        paths = []string{output}
    }
    return a.postTask(AgentTask{Action: "receive", Paths: paths})
}

func (a *App) Chat() (AgentStatus, error) {
    return a.postTask(AgentTask{Action: "chat"})
}

func (a *App) ChatSaveDirectory() (string, error) {
    dir, err := currentChatSaveDirectory()

```

```

    if err != nil {
        return "", err
    }
    if err := os.MkdirAll(dir, 0700); err != nil {
        return "", err
    }
    _ = cleanupOldChatSaveDirectories(chatSaveRetentionDays)
    return dir, nil
}

func (a *App) DownloadChatAttachment(rawURL string, filename string) (string, error) {
    parsed, err := chatAttachmentDownloadURL(rawURL)
    if err != nil {
        return "", err
    }
    dir, err := a.ChatSaveDirectory()
    if err != nil {
        return "", err
    }
    if filename == "" {
        filename = filepath.Base(parsed.Path)
    }
    target, err := uniquePath(dir, safeFilename(filename))
    if err != nil {
        return "", err
    }
    if err := a.downloadChatAttachmentTo(parsed.String(), target); err != nil {
        return "", err
    }
    return target, nil
}

func (a *App) SaveChatAttachmentAs(rawURL string, filename string) (string, error) {
    parsed, err := chatAttachmentDownloadURL(rawURL)
    if err != nil {
        return "", err
    }
    if filename == "" {
        filename = filepath.Base(parsed.Path)
    }
    target, err := wailsruntime.SaveFileDialog(a.ctx, wailsruntime.SaveDialogOptions{
        Title:      "Save attachment as",
        DefaultFilename: safeFilename(filename),
    })
    if err != nil {
        return "", err
    }
    if target == "" {
        return "", nil
    }
}

```

```

    if err := a.downloadChatAttachmentTo(parsed.String(), target); err != nil {
        return "", err
    }
    return target, nil
}

func chatAttachmentDownloadURL(rawURL string) (*url.URL, error) {
    parsed, err := url.Parse(rawURL)
    if err != nil {
        return nil, err
    }
    if parsed.Scheme != "http" && parsed.Scheme != "https" {
        return nil, fmt.Errorf("unsupported attachment URL scheme %q", parsed.Scheme)
    }
    query := parsed.Query()
    query.Set("download", "1")
    parsed.RawQuery = query.Encode()
    return parsed, nil
}

func (a *App) downloadChatAttachmentTo(rawURL string, target string) error {
    req, err := http.NewRequestWithContext(a.ctx, http.MethodGet, rawURL, nil)
    if err != nil {
        return err
    }
    resp, err := a.client.Do(req)
    if err != nil {
        return err
    }
    defer resp.Body.Close()
    if resp.StatusCode < 200 || resp.StatusCode > 299 {
        return fmt.Errorf("attachment download returned %s", resp.Status)
    }
    out, err := os.OpenFile(target, os.O_WRONLY|os.O_CREATE|os.O_TRUNC, 0600)
    if err != nil {
        return err
    }
    _, copyErr := io.Copy(out, resp.Body)
    closeErr := out.Close()
    if copyErr != nil {
        return copyErr
    }
    if closeErr != nil {
        return closeErr
    }
    return nil
}

func (a *App) StopCurrent() error {
    if err := a.ensureAgent(); err != nil {

```

```

    return err
}
return a.postNoBody("/stop-current")
}

func (a *App) ClearHistory() error {
    if err := a.ensureAgent(); err != nil {
        return err
    }
    req, err := http.NewRequestWithContext(a.ctx, http.MethodDelete, agentBaseURL+"/history", nil)
    if err != nil {
        return err
    }
    resp, err := a.client.Do(req)
    if err != nil {
        return err
    }
    defer resp.Body.Close()
    if resp.StatusCode < 200 || resp.StatusCode > 299 {
        return fmt.Errorf("desktop agent returned %s", resp.Status)
    }
    return nil
}

func (a *App) RepeatTask(id int) (AgentStatus, error) {
    if id <= 0 {
        return AgentStatus{}, fmt.Errorf("invalid task id")
    }
    if err := a.ensureAgent(); err != nil {
        return AgentStatus{}, err
    }
    var status AgentStatus
    if err := a.postJSON(fmt.Sprintf("/tasks/%d/repeat", id), nil, &status); err != nil {
        return AgentStatus{}, err
    }
    return status, nil
}

func (a *App) RestartAgent() error {
    if err := a.ensureAgent(); err != nil {
        return err
    }
    return a.postNoBody("/restart")
}

func (a *App) OpenURL(rawURL string) error {
    return a.openExternal(rawURL, map[string]bool{"http": true, "https": true})
}

func (a *App) OpenExternal(rawURL string) error {

```

```
    return a.openExternal(rawURL, map[string]bool{"http": true, "https": true, "mailto": true})
}
```

```
func (a *App) openExternal(rawURL string, allowed map[string]bool) error {
    parsed, err := url.Parse(rawURL)
    if err != nil {
        return err
    }
    if !allowed[parsed.Scheme] {
        return fmt.Errorf("unsupported URL scheme %q", parsed.Scheme)
    }
    wailsruntime.BrowserOpenURL(a.ctx, rawURL)
    return nil
}
```

```
func (a *App) OpenPath(path string) error {
    if path == "" {
        return fmt.Errorf("path is empty")
    }
    info, err := os.Stat(path)
    if err != nil {
        return err
    }
    target := path
    if !info.IsDir() {
        target = filepath.Dir(path)
    }
    cmd, err := openPathCommand(target)
    if err != nil {
        return err
    }
    return cmd.Start()
}
```

```
func (a *App) OpenFile(path string) error {
    if path == "" {
        return fmt.Errorf("path is empty")
    }
    info, err := os.Stat(path)
    if err != nil {
        return err
    }
    if info.IsDir() {
        return a.OpenPath(path)
    }
    cmd, err := openFileCommand(path)
    if err != nil {
        return err
    }
    return cmd.Start()
}
```

```

}

func (a *App) ReadSettings() (DesktopSettings, error) {
    if err := a.ensureAgent(); err != nil {
        return DesktopSettings{}, err
    }
    var settings DesktopSettings
    if err := a.getJSON("/settings", &settings); err != nil {
        return DesktopSettings{}, err
    }
    a.setCloseBehavior(settings.CloseBehavior)
    return settings, nil
}

func (a *App) SaveSettings(settings DesktopSettings) (DesktopSettings, error) {
    if err := a.ensureAgent(); err != nil {
        return DesktopSettings{}, err
    }
    var saved DesktopSettings
    if err := a.postJSON("/settings", settings, &saved); err != nil {
        return DesktopSettings{}, err
    }
    a.setCloseBehavior(saved.CloseBehavior)
    return saved, nil
}

func (a *App) SelectFiles() ([]string, error) {
    return wailsruntime.OpenMultipleFileDialog(a.ctx, wailsruntime.OpenDialogOptions{
        Title: "Choose files to share",
    })
}

func (a *App) SelectShareDirectory() (string, error) {
    return wailsruntime.OpenDirectoryDialog(a.ctx, wailsruntime.OpenDialogOptions{
        Title: "Choose folder to share",
    })
}

func (a *App) SelectReceiveDirectory() (string, error) {
    return wailsruntime.OpenDirectoryDialog(a.ctx, wailsruntime.OpenDialogOptions{
        Title: "Choose receive folder",
    })
}

func (a *App) AppInfo() AppInfo {
    info := AppInfo{
        Product:      "EQT",
        Name:         "Easy QR Transfer",
        Description:   "Local QR-code file transfer for desktop and mobile devices.",
        AgentURL:     agentBaseURL,
    }

```

```

    OS:          runtime.GOOS,
    Arch:        runtime.GOARCH,
}
if cli, err := findEqrcpCLI(); err == nil {
    info.CLIPath = cli
}
return info
}

func (a *App) postTask(task AgentTask) (AgentStatus, error) {
    if err := a.ensureAgent(); err != nil {
        return AgentStatus{}, err
    }
    var status AgentStatus
    if err := a.postJSON("/tasks", task, &status); err != nil {
        return AgentStatus{}, err
    }
    return status, nil
}

func (a *App) ensureAgent() error {
    if a.health() == nil {
        return nil
    }
    cli, err := findEqrcpCLI()
    if err != nil {
        return err
    }
    cmd := exec.Command(cli, "desktop", "agent-start", "-B")
    if launcher, ok := findEqrcpLauncher(cli); ok {
        cmd = exec.Command(launcher, "--eqrcp-exe", cli, "agent-start", "-B")
    }
    cmd.Stdout = os.Stdout
    cmd.Stderr = os.Stderr
    if err := cmd.Start(); err != nil {
        return fmt.Errorf("start desktop agent: %w", err)
    }
    if err := cmd.Wait(); err != nil {
        return fmt.Errorf("start desktop agent: %w", err)
    }
    deadline := time.Now().Add(5 * time.Second)
    for time.Now().Before(deadline) {
        if a.health() == nil {
            return nil
        }
        time.Sleep(150 * time.Millisecond)
    }
    return fmt.Errorf("desktop agent did not become ready")
}

```

```

func (a *App) health() error {
    req, err := http.NewRequestWithContext(a.ctx, http.MethodGet, agentBaseURL+"/health", nil)
    if err != nil {
        return err
    }
    resp, err := a.client.Do(req)
    if err != nil {
        return err
    }
    defer resp.Body.Close()
    if resp.StatusCode != http.StatusNoContent {
        return fmt.Errorf("desktop agent health returned %s", resp.Status)
    }
    return nil
}

```

```

func (a *App) getJSON(path string, out interface{}) error {
    req, err := http.NewRequestWithContext(a.ctx, http.MethodGet, agentBaseURL+path, nil)
    if err != nil {
        return err
    }
    resp, err := a.client.Do(req)
    if err != nil {
        return err
    }
    defer resp.Body.Close()
    if resp.StatusCode < 200 || resp.StatusCode > 299 {
        return fmt.Errorf("desktop agent returned %s", resp.Status)
    }
    return json.NewDecoder(resp.Body).Decode(out)
}

```

```

func (a *App) postJSON(path string, in interface{}, out interface{}) error {
    var body *bytes.Reader
    if in == nil {
        body = bytes.NewReader(nil)
    } else {
        data, err := json.Marshal(in)
        if err != nil {
            return err
        }
        body = bytes.NewReader(data)
    }
    req, err := http.NewRequestWithContext(a.ctx, http.MethodPost, agentBaseURL+path, body)
    if err != nil {
        return err
    }
    req.Header.Set("Content-Type", "application/json")
    resp, err := a.client.Do(req)
    if err != nil {

```

```

    return err
}
defer resp.Body.Close()
if resp.StatusCode < 200 || resp.StatusCode > 299 {
    return fmt.Errorf("desktop agent returned %s", resp.Status)
}
if out == nil {
    return nil
}
return json.NewDecoder(resp.Body).Decode(out)
}

func (a *App) postNoBody(path string) error {
    req, err := http.NewRequestWithContext(a.ctx, http.MethodPost, agentBaseURL+path, nil)
    if err != nil {
        return err
    }
    resp, err := a.client.Do(req)
    if err != nil {
        return err
    }
    defer resp.Body.Close()
    if resp.StatusCode < 200 || resp.StatusCode > 299 {
        return fmt.Errorf("desktop agent returned %s", resp.Status)
    }
    return nil
}

func findEqrcpCLI() (string, error) {
    if configured := os.Getenv("EQRCP_CLI"); configured != "" {
        return configured, nil
    }
    if exe, err := os.Executable(); err == nil {
        name := "eqrcp"
        if runtime.GOOS == "windows" {
            name = "eqrcp.exe"
        }
        candidate := filepath.Join(filepath.Dir(exe), name)
        if _, err := os.Stat(candidate); err == nil {
            return candidate, nil
        }
    }
    if path, err := exec.LookPath("eqrcp"); err == nil {
        return path, nil
    }
    return "", fmt.Errorf("eqrcp CLI was not found; set EQRCP_CLI or place eqrcp next to the desktop a
}

func findEqrcpLauncher(cli string) (string, bool) {
    if runtime.GOOS != "windows" || cli == "" {

```

```

    return "", false
}
candidate := filepath.Join(filepath.Dir(cli), "eqrcp-launcher.exe")
if _, err := os.Stat(candidate); err == nil {
    return candidate, true
}
return "", false
}

func openPathCommand(path string) (*exec.Cmd, error) {
    switch runtime.GOOS {
    case "windows":
        return exec.Command("explorer.exe", path), nil
    case "darwin":
        return exec.Command("open", path), nil
    case "linux":
        return exec.Command("xdg-open", path), nil
    default:
        return nil, fmt.Errorf("opening paths is not supported on %s", runtime.GOOS)
    }
}

func openFileCommand(path string) (*exec.Cmd, error) {
    switch runtime.GOOS {
    case "windows":
        return exec.Command("rundll32.exe", "url.dll,FileProtocolHandler", path), nil
    case "darwin":
        return exec.Command("open", path), nil
    case "linux":
        return exec.Command("xdg-open", path), nil
    default:
        return nil, fmt.Errorf("opening files is not supported on %s", runtime.GOOS)
    }
}

func currentChatSaveDirectory() (string, error) {
    root, err := chatSaveRoot()
    if err != nil {
        return "", err
    }
    return filepath.Join(root, time.Now().Format("2006-01-02")), nil
}

func chatSaveRoot() (string, error) {
    return filepath.Join(os.TempDir(), "EQT Chat"), nil
}

func cleanupOldChatSaveDirectories(retentionDays int) error {
    root, err := chatSaveRoot()
    if err != nil {

```

```

    return err
}
entries, err := os.ReadDir(root)
if err != nil {
    if os.IsNotExist(err) {
        return nil
    }
    return err
}
cutoff := time.Now().AddDate(0, 0, -retentionDays)
for _, entry := range entries {
    if !entry.IsDir() {
        continue
    }
    day, err := time.Parse("2006-01-02", entry.Name())
    if err != nil || !day.Before(cutoff) {
        continue
    }
    _ = os.RemoveAll(filepath.Join(root, entry.Name()))
}
return nil
}

func safeFilename(name string) string {
    name = strings.TrimSpace(filepath.Base(name))
    if name == "" || name == "." || name == string(filepath.Separator) {
        return "attachment"
    }
    replacer := strings.NewReplacer("/", "_", "\\", "_", ":", "_", "*", "_", "?", "_", "'", "_", "<",
    name = replacer.Replace(name)
    if len([]rune(name)) > 160 {
        ext := filepath.Ext(name)
        base := strings.TrimSuffix(name, ext)
        runes := []rune(base)
        limit := 160 - len([]rune(ext))
        if limit < 1 {
            limit = 1
        }
        if len(runes) > limit {
            base = string(runes[:limit])
        }
        name = base + ext
    }
    return name
}

func uniquePath(dir string, name string) (string, error) {
    for index := 0; ; index++ {
        candidate := name
        if index > 0 {

```

```

    ext := filepath.Ext(name)
    base := strings.TrimSuffix(name, ext)
    candidate = fmt.Sprintf("%s(%d)%s", base, index, ext)
}
path := filepath.Join(dir, candidate)
if _, err := os.Stat(path); err != nil {
    if os.IsNotExist(err) {
        return path, nil
    }
    return "", err
}
}
}
package main

import (
    "embed"
    "log"
    "sync"

    "fyne.io/systray"
)

//go:embed frontend/src/assets/images/logo-universal.png
var trayAssets embed.FS

type trayController struct {
    app    *App
    start func()
    end    func()
    once   sync.Once
}

func newTrayController(app *App) *trayController {
    tray := &trayController{app: app}
    tray.start, tray.end = systray.RunWithExternalLoop(tray.onReady, tray.onExit)
    return tray
}

func (t *trayController) startTray() {
    if t == nil || t.start == nil {
        return
    }
    t.once.Do(t.start)
}

func (t *trayController) shutdown() {
    if t == nil || t.end == nil {
        return
    }
}

```

```

    t.end()
}

func (t *trayController) onReady() {
    icon, err := trayAssets.ReadFile("frontend/src/assets/images/logo-universal.png")
    if err == nil {
        systray.SetIcon(icon)
    } else {
        log.Printf("load tray icon: %v", err)
    }
    systray.SetTitle("EQT")
    systray.SetTooltip("EQT - Easy QR Transfer")

    open := systray.AddMenuItem("Open EQT", "Show the EQT window")
    share := systray.AddMenuItem("Share...", "Open the share workflow")
    receive := systray.AddMenuItem("Receive...", "Open the receive workflow")
    systray.AddSeparator()
    openCurrent := systray.AddMenuItem("Open Current QR", "Open the active transfer QR page")
    stopCurrent := systray.AddMenuItem("Stop Current Transfer", "Stop the active transfer")
    systray.AddSeparator()
    settings := systray.AddMenuItem("Settings", "Open EQT settings")
    about := systray.AddMenuItem("About EQT", "Show product information")
    feedback := systray.AddMenuItem("Send Feedback", "Open the feedback form")
    systray.AddSeparator()
    quit := systray.AddMenuItem("Quit", "Quit EQT")

    systray.SetOnTapped(func() {
        t.showAndEmit("")
    })
    go t.handle(open, func() { t.showAndEmit("") })
    go t.handle(share, func() { t.showAndEmit("share") })
    go t.handle(receive, func() { t.showAndEmit("receive") })
    go t.handle(openCurrent, t.openCurrentQR)
    go t.handle(stopCurrent, t.stopCurrent)
    go t.handle(settings, func() { t.showAndEmit("settings") })
    go t.handle(about, func() { t.showAndEmit("about") })
    go t.handle(feedback, func() { t.showAndEmit("feedback") })
    go t.handle(quit, t.quit)
}

func (t *trayController) onExit() {}

func (t *trayController) handle(item *systray.MenuItem, fn func()) {
    for range item.ClickedCh {
        fn()
    }
}

func (t *trayController) showAndEmit(command string) {
    t.app.showWindow()
}

```

```

    if command != "" {
        t.app.emitTrayCommand(command)
    }
}

func (t *trayController) openCurrentQR() {
    t.app.showWindow()
    status, err := t.app.AgentStatus()
    if err != nil {
        t.app.emitTrayCommand("refresh")
        return
    }
    if status.Current == nil || status.Current.PageURL == "" {
        t.app.emitTrayCommand("refresh")
        return
    }
    if err := t.app.OpenURL(status.Current.PageURL); err != nil {
        t.app.emitTrayCommand("refresh")
    }
}

func (t *trayController) stopCurrent() {
    t.app.showWindow()
    if err := t.app.StopCurrent(); err != nil {
        t.app.emitTrayCommand("refresh")
        return
    }
    t.app.emitTrayCommand("refresh")
}

func (t *trayController) quit() {
    systray.Quit()
    t.app.quit()
}

// gen-softcopy-pdf generates the source code PDF for software copyright (t%fl% ) application.
// Usage: go run scripts/gen-softcopy-pdf.go
// Output: t%fl% ç t%fl%æ æ /eqrcp-source-code-V1.0.pdf

package main

import (
    "bufio"
    "bytes"
    "fmt"
    "os"
    "path/filepath"
    "sort"
    "strings"
)

```

```

const (
    softwareName = "eqrcp V1.0"
    linesPerPage = 50
    targetLines  = linesPerPage * 30 // 1500 lines for 30 pages
    outputPath   = "tflk ç tflæ æ /eqrcp-source-code-V1.0.pdf"
    pageWidth    = 595.0
    pageHeight   = 842.0
    marginLeft   = 50.0
    marginTop    = 50.0
    headerFontSize = 8.0
    codeFontSize  = 7.0
    lineHeight   = 9.5
    charsPerLine = 100
)

type FileInfo struct {
    Path string
    Lines int
}

func main() {
    projectRoot, err := findProjectRoot()
    if err != nil {
        fmt.Fprintf(os.Stderr, "Error: %v\n", err)
        os.Exit(1)
    }
    os.Chdir(projectRoot)

    var files []FileInfo
    filepath.Walk(".", func(path string, info os.FileInfo, err error) error {
        if err != nil || info.IsDir() {
            return nil
        }
        if !strings.HasSuffix(path, ".go") || strings.HasSuffix(info.Name(), "_test.go") {
            return nil
        }
        lines, err := countLines(path)
        if err != nil {
            return nil
        }
        files = append(files, FileInfo{Path: path, Lines: lines})
        return nil
    })

    sort.Slice(files, func(i, j int) bool {
        return fileOrder(files[i].Path) < fileOrder(files[j].Path)
    })

    firstLines := extractLines(files, targetLines)
    lastLines := extractLinesReverse(files, targetLines)

```

```

var pages []string
allLines := append(firstLines, lastLines...)

for i := 0; i < len(allLines); i += linesPerPage {
    end := i + linesPerPage
    if end > len(allLines) {
        end = len(allLines)
    }
    pageNum := len(pages) + 1
    pageContent := buildPageContent(allLines[i:end], pageNum)
    pages = append(pages, pageContent)
}

pdfData := buildPDF(pages)
if err := os.MkdirAll(filepath.Dir(outputPath), 0755); err != nil {
    fmt.Fprintf(os.Stderr, "Error creating output dir: %v\n", err)
    os.Exit(1)
}
if err := os.WriteFile(outputPath, pdfData, 0644); err != nil {
    fmt.Fprintf(os.Stderr, "Error writing PDF: %v\n", err)
    os.Exit(1)
}
fmt.Printf("Generated %s (%d pages total: %d front + %d back, %d lines)\n",
    outputPath, len(pages), len(pages)/2, len(pages)-len(pages)/2, len(allLines))
}

func findProjectRoot() (string, error) {
    exe, _ := os.Executable()
    for dir := filepath.Dir(exe); dir != "/" && dir != "."; dir = filepath.Dir(dir) {
        if _, err := os.Stat(filepath.Join(dir, "go.mod")); err == nil {
            return dir, nil
        }
    }
    return os.Getwd()
}

func countLines(path string) (int, error) {
    f, err := os.Open(path)
    if err != nil {
        return 0, err
    }
    defer f.Close()
    scanner := bufio.NewScanner(f)
    count := 0
    for scanner.Scan() {
        count++
    }
    return count, scanner.Err()
}

```

```

func fileOrder(path string) int {
    order := map[string]int{
        "main.go": 0,
        "application/application.go": 1,
        "cmd/qrcp.go": 10,
        "cmd/send.go": 11,
        "cmd/receive.go": 12,
        "cmd/chat.go": 13,
        "cmd/config.go": 14,
        "cmd/version.go": 15,
        "cmd/completion.go": 16,
        "cmd/desktop.go": 17,
        "cmd/desktop_agent.go": 18,
        "cmd/desktop_integration.go": 19,
        "cmd/desktop_agent_background_windows.go": 20,
        "cmd/desktop_agent_background_other.go": 21,
        "cmd/eqrcp-launcher/main.go": 25,
        "cmd/eqrcp-launcher/launcher_windows.go": 26,
        "cmd/eqrcp-launcher/launcher_other.go": 27,
        "config/config.go": 30,
        "config/settings.go": 31,
        "config/migrate.go": 32,
        "config/util.go": 33,
        "body/payload.go": 40,
        "server/server.go": 50,
        "server/chat.go": 51,
        "server/util.go": 52,
        "server/tcpkeepalivelistener.go": 53,
        "qr/qr.go": 60,
        "util/util.go": 70,
        "util/net.go": 71,
        "logger/logger.go": 80,
        "pages/pages.go": 90,
        "version/version.go": 100,
        "desktop/gui/main.go": 110,
        "desktop/gui/app.go": 111,
        "desktop/gui/tray.go": 112,
    }
    if o, ok := order[path]; ok {
        return o
    }
    return 1000
}

func readFileLines(path string) ([]string, error) {
    f, err := os.Open(path)
    if err != nil {
        return nil, err
    }

```

```

defer f.Close()
var lines []string
scanner := bufio.NewScanner(f)
for scanner.Scan() {
    lines = append(lines, scanner.Text())
}
return lines, scanner.Err()
}

func extractLines(files []FileInfo, target int) []string {
var result []string
for _, f := range files {
    if len(result) >= target {
        break
    }
    lines, err := readFileLines(f.Path)
    if err != nil {
        continue
    }
    needed := target - len(result)
    if len(lines) > needed {
        result = append(result, lines[:needed]...)
    } else {
        result = append(result, lines...)
    }
}
return result
}

func extractLinesReverse(files []FileInfo, target int) []string {
var result []string
for i := len(files) - 1; i >= 0; i-- {
    if len(result) >= target {
        break
    }
    lines, err := readFileLines(files[i].Path)
    if err != nil {
        continue
    }
    needed := target - len(result)
    if len(lines) > needed {
        result = append(append([]string{}, lines[len(lines)-needed:]), result...)
    } else {
        result = append(append([]string{}, lines...), result...)
    }
}
return result
}

func buildPageContent(lines []string, pageNum int) string {

```

```

header := fmt.Sprintf("%s â Page %d", softwareName, pageNum)
codeStartY := pageHeight - marginTop - 14

var buf bytes.Buffer

// Code lines
buf.WriteString("BT\n")
buf.WriteString(fmt.Sprintf("/F1 %.1f Tf\n", codeFontSize))
buf.WriteString(fmt.Sprintf("%.1f TL\n", lineHeight))
for i, line := range lines {
    y := codeStartY - float64(i)*lineHeight
    if y < 40 {
        break
    }
    escaped := escapePDFString(line)
    if len(escaped) > charsPerLine {
        escaped = escaped[:charsPerLine]
    }
    buf.WriteString(fmt.Sprintf("1 0 0 1 %.1f %.1f Tm (%s) Tj\n", marginLeft, y, escaped))
}
buf.WriteString("ET\n")

// Page header
buf.WriteString("BT\n")
buf.WriteString(fmt.Sprintf("/F1 %.1f Tf\n", headerFontSize))
headerY := pageHeight - marginTop + 6
buf.WriteString(fmt.Sprintf("1 0 0 1 %.1f %.1f Tm (%s) Tj\n", marginLeft, headerY, escapePDFSt
buf.WriteString("ET\n")

return buf.String()
}

func escapePDFString(s string) string {
    s = strings.ReplaceAll(s, "\\\"", "\\\"")
    s = strings.ReplaceAll(s, "(", "\\(")
    s = strings.ReplaceAll(s, ")", "\\)")
    return s
}

// buildPDF generates a PDF 1.4 document using only built-in Courier font.
func buildPDF(pages []string) []byte {
    var buf bytes.Buffer
    var offsets []int

    type pdfObj struct {
        num int
        data string
    }
    var allObjs []pdfObj
    objNum := 0

```

```

nextObj := func(data string) int {
    objNum++
    allObjs = append(allObjs, pdfObj{objNum, data})
    return objNum
}

fontRef := nextObj("<< /Type /Font /Subtype /Type1 /BaseFont /Courier >>")

var contentRefs []int
for _, cs := range pages {
    contentRefs = append(contentRefs, nextObj(
        fmt.Sprintf("<< /Length %d >>\nstream\n%s\nendstream", len(cs), cs)))
}

var pageRefs []int
for _, cr := range contentRefs {
    pageRefs = append(pageRefs, nextObj(
        fmt.Sprintf("<< /Type /Page /Parent 3 0 R /MediaBox [0 0 %.1f %.1f] /Contents %d 0 R /Resources\n    pageWidth, pageHeight, cr, fontRef)))
}

kids := ""
for _, p := range pageRefs {
    kids += fmt.Sprintf("%d 0 R ", p)
}

```